МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №7

Специальность ПО-13

Выполнил:

Д. А. Марковский

студент группы ПО-13

Проверил:

А. А. Крощенко

Доцент кафедры ИИТ

27.03.2026

Брест 2026

Цель работы:

Освоить возможности языка программирования Python в разработке оконных приложений.

Задание 1: Построение графических примитивов и надписей

• Реализовать соответствующие классы, указанные в задании;
• Организовать ввод параметров для создания объектов (использовать экранные компоненты);
• Осуществить визуализацию графических примитивов
Важное замечание: должна быть предусмотрена возможность приостановки выполнения визуализации, изменения параметров «на лету» и снятия скриншотов с сохранением в текущую активную директорию. Для всех динамических сцен необходимо задавать параметр скорости!
5) Изобразить в окне приложения отрезок, вращающийся в плоскости экрана вокруг одной из своих концевых точек. Цвет прямой должен изменяться при переходе от одного положения к другому.

Код программы:

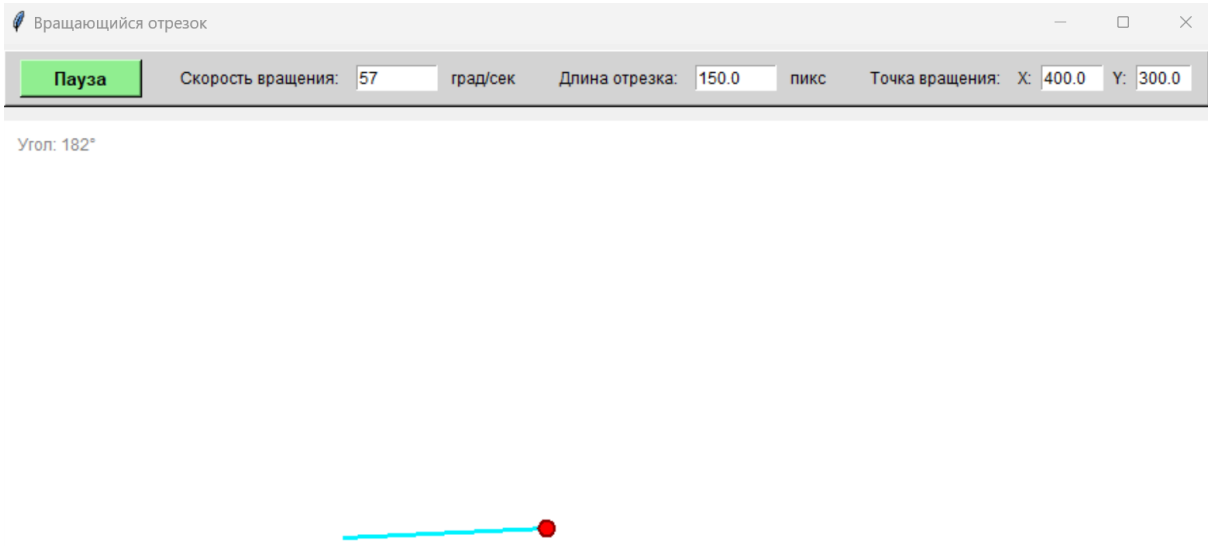
```
import time
import tkinter as tk
import math
import colorsys
from tkinter import messagebox
from PIL import ImageGrab


# pylint: disable=too-many-instance-attributes
class RotatingLineApp:
    def __init__(self, root):
        self.canvas = None
        self.screenshot_btn = None
        self.y_entry = None
        self.y_var = None
        self.x_entry = None
        self.x_var = None
        self.length_entry = None
        self.length_var = None
        self.speed_entry = None
        self.speed_var = None
        self.play_pause_btn = None

        self.root = root
        self.root.title("Вращающийся отрезок")
        self.root.geometry("900x700")
        self.root.minsize(600, 500)

        self.length = 150.0
        self.fixed_x = 400.0
        self.fixed_y = 300.0
        self.angle = 0.0
        self.angular_speed = 0.05
        self.running = False
        self.after_id = None

        self.create_widgets()
        self.canvas.bind('<Configure>', self.on_canvas_resize)
```

```
        self.redraw()

    def create_widgets(self):
        control_frame = tk.Frame(self.root, bg='lightgray', relief=tk.RAISED,
bd=2)
        control_frame.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)

        btn_frame = tk.Frame(control_frame, bg='lightgray')
        btn_frame.pack(side=tk.LEFT, padx=10, pady=5)

        speed_frame = tk.Frame(control_frame, bg='lightgray')
        speed_frame.pack(side=tk.LEFT, padx=10, pady=5)

        length_frame = tk.Frame(control_frame, bg='lightgray')
        length_frame.pack(side=tk.LEFT, padx=10, pady=5)

        pos_frame = tk.Frame(control_frame, bg='lightgray')
        pos_frame.pack(side=tk.LEFT, padx=10, pady=5)

        screenshot_frame = tk.Frame(control_frame, bg='lightgray')
        screenshot_frame.pack(side=tk.RIGHT, padx=10, pady=5)

        self.play_pause_btn = tk.Button(btn_frame, text="▶ Старт",
                                        command=self.toggle_animation,
                                        width=10, height=1, font=('Arial', 10,
'bold'))
        self.play_pause_btn.pack()

        # Скорость вращения (градусы/сек)
        tk.Label(speed_frame, text="Скорость вращения:", bg='lightgray',
                 font=('Arial', 9)).pack(side=tk.LEFT, padx=5)
        self.speed_var = tk.StringVar(
            value=str(int(self.angular_speed * (1000 / 50) * 180 / math.pi)))
        self.speed_entry = tk.Entry(speed_frame, textvariable=self.speed_var,
                                    width=8, font=('Arial', 9))
        self.speed_entry.pack(side=tk.LEFT, padx=5)
        self.speed_var.trace('w', lambda *args: self.update_speed())
        tk.Label(speed_frame, text="град/сек", bg='lightgray',
                 font=('Arial', 9)).pack(side=tk.LEFT, padx=2)

        # Длина отрезка
        tk.Label(length_frame, text="Длина отрезка:", bg='lightgray',
                 font=('Arial', 9)).pack(side=tk.LEFT, padx=5)
        self.length_var = tk.StringVar(value=str(self.length))
        self.length_entry = tk.Entry(length_frame, textvariable=self.length_var,
                                     width=8, font=('Arial', 9))
        self.length_entry.pack(side=tk.LEFT, padx=5)
        self.length_var.trace('w', lambda *args: self.update_length())
        tk.Label(length_frame, text="пикс", bg='lightgray',
                 font=('Arial', 9)).pack(side=tk.LEFT, padx=2)

        tk.Label(pos_frame, text="Точка вращения:", bg='lightgray',
                 font=('Arial', 9)).pack(side=tk.LEFT, padx=5)

        tk.Label(pos_frame, text="X:", bg='lightgray',
                 font=('Arial', 9)).pack(side=tk.LEFT, padx=2)
        self.x_var = tk.StringVar(value=str(self.fixed_x))
        self.x_entry = tk.Entry(pos_frame, textvariable=self.x_var,
                                width=6, font=('Arial', 9))
        self.x_entry.pack(side=tk.LEFT, padx=2)
        self.x_var.trace('w', lambda *args: self.update_fixed_point())
```

```
        tk.Label(pos_frame, text="Y:", bg='lightgray',
                 font=('Arial', 9)).pack(side=tk.LEFT, padx=2)
        self.y_var = tk.StringVar(value=str(self.fixed_y))
        self.y_entry = tk.Entry(pos_frame, textvariable=self.y_var,
                                width=6, font=('Arial', 9))
        self.y_entry.pack(side=tk.LEFT, padx=2)
        self.y_var.trace('w', lambda *args: self.update_fixed_point())

        self.screenshot_btn = tk.Button(screenshot_frame, text="Скриншот",
                                        command=self.take_screenshot,
                                        width=12, height=1, font=('Arial', 9))
        self.screenshot_btn.pack()
        self.canvas = tk.Canvas(self.root, bg="white", highlightthickness=0)
        self.canvas.pack(side=tk.BOTTOM, fill=tk.BOTH, expand=True, padx=5,
pady=5)

    def on_canvas_resize(self, _):
        self.redraw()

    def toggle_animation(self):
        if self.running:
            self.running = False
            self.play_pause_btn.config(text="Старт", bg='SystemButtonFace')
            if self.after_id:
                self.root.after_cancel(self.after_id)
                self.after_id = None
        else:
            self.running = True
            self.play_pause_btn.config(text="Пауза", bg='lightgreen')
            self.animate()

    def animate(self):
        if not self.running:
            return
        self.angle += self.angular_speed
        self.redraw()
        self.after_id = self.root.after(50, self.animate)

    def redraw(self):
        self.canvas.delete("all")
        canvas_width = self.canvas.winfo_width()
        canvas_height = self.canvas.winfo_height()

        if canvas_width <= 1 or canvas_height <= 1:
            return

        if (self.fixed_x < 0 or self.fixed_x > canvas_width or
                self.fixed_y < 0 or self.fixed_y > canvas_height):
            self.canvas.create_text(canvas_width // 2, canvas_height // 2,
                                    text="Точка вращения вне области видимости!",
                                    fill="red", font=('Arial', 14, 'bold'))
            return

        end_x = self.fixed_x + self.length * math.cos(self.angle)
        end_y = self.fixed_y - self.length * math.sin(self.angle)
        color = self.angle_to_color(self.angle)
        self.canvas.create_line(self.fixed_x, self.fixed_y, end_x, end_y,
                                fill=color, width=3)
        self.canvas.create_oval(self.fixed_x - 6, self.fixed_y - 6,
                                self.fixed_x + 6, self.fixed_y + 6,
                                fill="red", outline="darkred", width=2)
```

```
        angle_deg = int(self.angle * 180 / math.pi) % 360
        self.canvas.create_text(10, 10, text=f"Угол: {angle_deg}°",
                                anchor=tk.NW, fill="gray", font=('Arial', 9))

    @staticmethod
    def angle_to_color(angle):
        hue = angle % (2 * math.pi)
        hue /= (2 * math.pi)
        r, g, b = colorsys.hsv_to_rgb(hue, 1.0, 1.0)
        return f'#{int(r * 255):02x}{int(g * 255):02x}{int(b * 255):02x}'

    def update_speed(self):
        try:
            value = self.speed_var.get().strip()
            if value:
                deg_per_sec = float(value)
                self.angular_speed = (deg_per_sec * math.pi / 180) * 0.05
        except ValueError:
            current_deg = self.angular_speed * (1000 / 50) * 180 / math.pi
            self.speed_var.set(str(int(current_deg)))
            messagebox.showerror("Ошибка",
                                 "Введите корректное число для скорости.")

    def update_length(self):
        try:
            value = self.length_var.get().strip()
            if value:
                new_len = float(value)
                if new_len > 0:
                    self.length = new_len
                    self.redraw()
                else:
                    raise ValueError
        except ValueError:
            self.length_var.set(str(self.length))
            messagebox.showerror("Ошибка",
                                 "Длина должна быть положительным числом.")

    def update_fixed_point(self):
        try:
            x_value = self.x_var.get().strip()
            y_value = self.y_var.get().strip()

            if x_value and y_value:
                x = float(x_value)
                y = float(y_value)

                if x >= 0 and y >= 0:
                    self.fixed_x = x
                    self.fixed_y = y
                    self.redraw()
                else:
                    raise ValueError
        except ValueError:
            self.x_var.set(str(self.fixed_x))
            self.y_var.set(str(self.fixed_y))
            messagebox.showerror("Ошибка",
                                 "Введите корректные положительные координаты.")

    def take_screenshot(self):
        try:
            x = self.root.winfo_rootx() + self.canvas.winfo_x()
```

```
            y = self.root.winfo_rooty() + self.canvas.winfo_y()
            w = self.canvas.winfo_width()
            h = self.canvas.winfo_height()

            if w <= 1 or h <= 1:
                messagebox.showwarning("Предупреждение",
                                       "Canvas еще не инициализирован.")
                return

            img = ImageGrab.grab(bbox=(x, y, x + w, y + h))
            timestamp = time.strftime("%Y%m%d_%H%M%S")
            filename = f"screenshot_{timestamp}.png"
            img.save(filename)
            messagebox.showinfo("Скриншот",
                                f"Сохранён как {filename}")
        except (OSError, IOError, AttributeError) as e:
            messagebox.showerror("Ошибка",
                                 f"Не удалось создать скриншот: {str(e)}")


if __name__ == "__main__":
    tk_root = tk.Tk()
    app = RotatingLineApp(tk_root)
    tk_root.mainloop()
```



Задание 2: Реализовать построение заданного типа фрактала по варианту

Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала.

5) Дерево Пифагора.

Код программы:

```
import time
import tkinter as tk
import math
from tkinter import messagebox
from PIL import ImageGrab


# pylint: disable=too-many-instance-attributes
class PythagorasTreeApp:
    def __init__(self, root_window):
        self.root = root_window
        self.root.title("Дерево Пифагора")
        self.root.geometry("1000x750")
        self.root.minsize(800, 600)

        self.start_x = 500.0
        self.start_y = 400.0
        self.base_length = 120.0
        self.angle_left = 45.0
        self.angle_right = 45.0
        self.shrink_factor = 0.7
        self.depth = 8

        self.animate = False
        self.current_depth = 1
        self.animation_speed = 100
        self.after_id = None
        self.max_depth = self.depth

        self.canvas = None
        self.x_scale = None
        self.y_scale = None
        self.length_scale = None
        self.left_angle_scale = None
        self.right_angle_scale = None
        self.shrink_scale = None
        self.shrink_label = None
        self.depth_scale = None
        self.depth_label = None
        self.animate_btn = None
        self.speed_scale = None
        self.screenshot_btn = None

        self.create_widgets()
        self.canvas.bind("<Configure>", lambda e: self.draw_tree())
        self.draw_tree()

    def create_widgets(self):
        control_frame = self._create_control_frame()
        self._create_row1(control_frame)
        self._create_row2(control_frame)
        self._create_row3(control_frame)
        self._create_row4(control_frame)
        self._create_canvas()
```

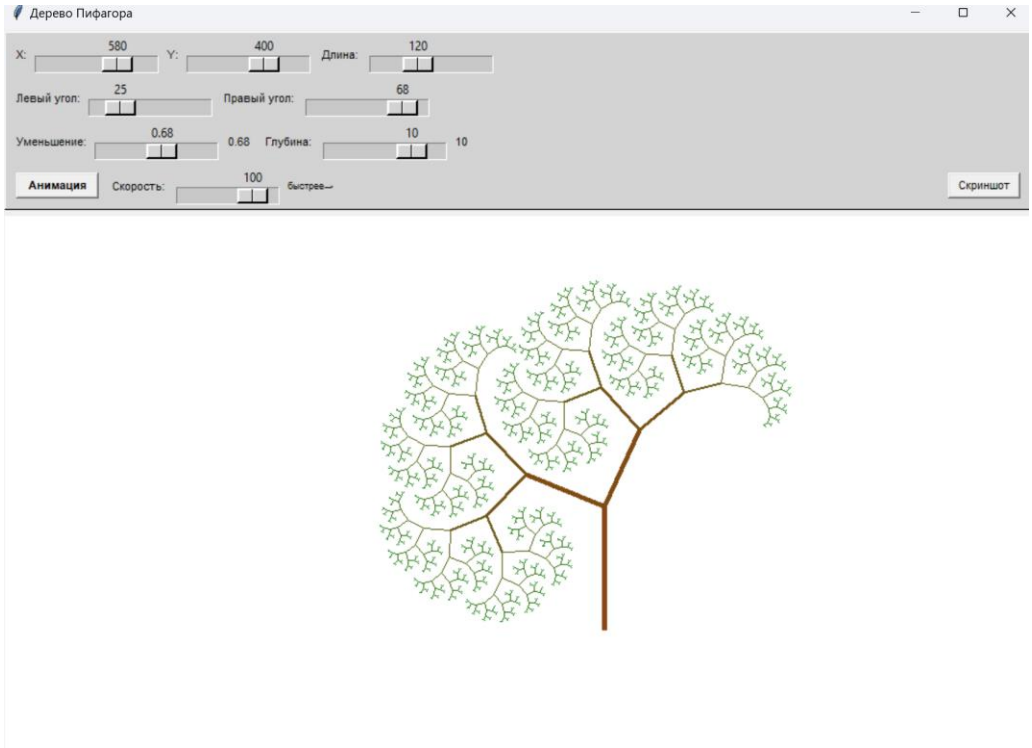
```
def _create_control_frame(self):
    frame = tk.Frame(self.root, bg="lightgray", relief=tk.RAISED, bd=1)
    frame.pack(side=tk.TOP, fill=tk.X, padx=3, pady=3)
    return frame

def _create_row1(self, parent):
    row = tk.Frame(parent, bg="lightgray")
    row.pack(fill=tk.X, padx=5, pady=2)

    tk.Label(row, text="X:", bg="lightgray", font=("Arial", 8)) \
        .pack(side=tk.LEFT, padx=2)
    self.x_scale = tk.Scale(
        row, from_=0, to=800, resolution=5, orient=tk.HORIZONTAL,
        length=120, command=self.update_start_x, bg="lightgray",
        highlightthickness=0
    )
    self.x_scale.set(self.start_x)
    self.x_scale.pack(side=tk.LEFT, padx=2)

    tk.Label(row, text="Y:", bg="lightgray", font=("Arial", 8)) \
        .pack(side=tk.LEFT, padx=2)
    self.y_scale = tk.Scale(
        row, from_=0, to=600, resolution=5, orient=tk.HORIZONTAL,
        length=120, command=self.update_start_y, bg="lightgray",
        highlightthickness=0
    )
    self.y_scale.set(self.start_y)
    self.y_scale.pack(side=tk.LEFT, padx=2)

    tk.Label(row, text="Длина:", bg="lightgray", font=("Arial", 8)) \
        .pack(side=tk.LEFT, padx=5)
    self.length_scale = tk.Scale(
        row, from_=50, to=250, resolution=5, orient=tk.HORIZONTAL,
        length=120, command=self.update_base_length, bg="lightgray",
        highlightthickness=0
    )
    self.length_scale.set(self.base_length)
    self.length_scale.pack(side=tk.LEFT, padx=2)

def _create_row2(self, parent):
    row = tk.Frame(parent, bg="lightgray")
    row.pack(fill=tk.X, padx=5, pady=2)

    tk.Label(row, text="Левый угол:", bg="lightgray", font=("Arial", 8)) \
        .pack(side=tk.LEFT, padx=2)
    self.left_angle_scale = tk.Scale(
        row, from_=15, to=75, resolution=1, orient=tk.HORIZONTAL,
        length=120, command=self.update_left_angle, bg="lightgray",
        highlightthickness=0
    )
    self.left_angle_scale.set(self.angle_left)
    self.left_angle_scale.pack(side=tk.LEFT, padx=2)

    tk.Label(row, text="Правый угол:", bg="lightgray", font=("Arial", 8)) \
        .pack(side=tk.LEFT, padx=5)
    self.right_angle_scale = tk.Scale(
        row, from_=15, to=75, resolution=1, orient=tk.HORIZONTAL,
        length=120, command=self.update_right_angle, bg="lightgray",
        highlightthickness=0
    )
    self.right_angle_scale.set(self.angle_right)
```

```
        self.right_angle_scale.pack(side=tk.LEFT, padx=2)

    def _create_row3(self, parent):
        row = tk.Frame(parent, bg="lightgray")
        row.pack(fill=tk.X, padx=5, pady=2)

        tk.Label(row, text="Уменьшение:", bg="lightgray", font=("Arial", 8)) \
            .pack(side=tk.LEFT, padx=2)
        self.shrink_scale = tk.Scale(
            row, from_=0.4, to=0.9, resolution=0.02, orient=tk.HORIZONTAL,
            length=120, command=self.update_shrink_factor, bg="lightgray",
            highlightthickness=0
        )
        self.shrink_scale.set(self.shrink_factor)
        self.shrink_scale.pack(side=tk.LEFT, padx=2)

        self.shrink_label = tk.Label(
            row, text=f"{self.shrink_factor:.2f}", bg="lightgray",
            width=4, font=("Arial", 8)
        )
        self.shrink_label.pack(side=tk.LEFT, padx=2)

        tk.Label(row, text="Глубина:", bg="lightgray", font=("Arial", 8)) \
            .pack(side=tk.LEFT, padx=5)
        self.depth_scale = tk.Scale(
            row, from_=2, to=12, resolution=1, orient=tk.HORIZONTAL,
            length=120, command=self.update_depth, bg="lightgray",
            highlightthickness=0
        )
        self.depth_scale.set(self.depth)
        self.depth_scale.pack(side=tk.LEFT, padx=2)

        self.depth_label = tk.Label(
            row, text=str(self.depth), bg="lightgray",
            width=2, font=("Arial", 8)
        )
        self.depth_label.pack(side=tk.LEFT, padx=2)

    def _create_row4(self, parent):
        row = tk.Frame(parent, bg="lightgray")
        row.pack(fill=tk.X, padx=5, pady=3)

        self.animate_btn = tk.Button(
            row, text="Анимация", command=self.toggle_animation,
            width=10, height=1, font=("Arial", 8, "bold")
        )
        self.animate_btn.pack(side=tk.LEFT, padx=5)

        tk.Label(row, text="Скорость:", bg="lightgray", font=("Arial", 8)) \
            .pack(side=tk.LEFT, padx=5)
        self.speed_scale = tk.Scale(
            row, from_=500, to=30, orient=tk.HORIZONTAL,
            length=100, command=self.update_animation_speed,
            bg="lightgray", highlightthickness=0
        )
        self.speed_scale.set(self.animation_speed)
        self.speed_scale.pack(side=tk.LEFT, padx=2)

        tk.Label(row, text="быстрее→", bg="lightgray", font=("Arial", 7)) \
            .pack(side=tk.LEFT, padx=2)

        self.screenshot_btn = tk.Button(
```

```
            row, text="Скриншот", command=self.take_screenshot,
            width=10, height=1, font=("Arial", 8)
        )
        self.screenshot_btn.pack(side=tk.RIGHT, padx=5)

    def _create_canvas(self):
        self.canvas = tk.Canvas(self.root, bg="white", highlightthickness=0)
        self.canvas.pack(side=tk.BOTTOM, fill=tk.BOTH, expand=True, padx=3,
pady=3)

    def draw_tree(self):
        self.canvas.delete("all")
        canvas_width = self.canvas.winfo_width()
        canvas_height = self.canvas.winfo_height()

        if canvas_width <= 1 or canvas_height <= 1:
            return

        self.max_depth = self.current_depth if self.animate else self.depth
        self.draw_branch((self.start_x, self.start_y), self.base_length, -90, 0)

    def _get_branch_style(self, depth):
        if depth == 0:
            return "#8b4513", 5

        ratio = min(1.0, depth / self.max_depth) if self.max_depth > 0 else 1.0
        r = int(139 + (34 - 139) * ratio)
        g = int(69 + (139 - 69) * ratio)
        b = int(19 + (34 - 19) * ratio)
        color = f"#{r:02x}{g:02x}{b:02x}"
        line_width = max(1, 5 - depth) if depth < 5 else 1
        return color, line_width

    def draw_branch(self, point, length, angle_deg, depth=0):
        if depth > self.max_depth or length < 1:
            return

        x, y = point
        angle_rad = math.radians(angle_deg)
        end_x = x + length * math.cos(angle_rad)
        end_y = y + length * math.sin(angle_rad)

        color, line_width = self._get_branch_style(depth)
        self.canvas.create_line(x, y, end_x, end_y, fill=color, width=line_width)

        new_length = length * self.shrink_factor
        self.draw_branch((end_x, end_y), new_length, angle_deg + self.angle_left,
depth + 1)
        self.draw_branch((end_x, end_y), new_length, angle_deg -
self.angle_right, depth + 1)

    def update_start_x(self, value):
        self.start_x = float(value)
        self.draw_tree()

    def update_start_y(self, value):
        self.start_y = float(value)
        self.draw_tree()

    def update_base_length(self, value):
        self.base_length = float(value)
        self.draw_tree()
```

```
    def update_left_angle(self, value):
        self.angle_left = float(value)
        self.draw_tree()

    def update_right_angle(self, value):
        self.angle_right = float(value)
        self.draw_tree()

    def update_shrink_factor(self, value):
        self.shrink_factor = float(value)
        self.shrink_label.config(text=f"{self.shrink_factor:.2f}")
        self.draw_tree()

    def update_depth(self, value):
        self.depth = int(value)
        self.depth_label.config(text=str(self.depth))
        self.current_depth = min(self.current_depth, self.depth)
        if not self.animate:
            self.draw_tree()

    def toggle_animation(self):
        if self.animate:
            self.animate = False
            self.animate_btn.config(text="Анимация", bg="SystemButtonFace")
            if self.after_id:
                self.root.after_cancel(self.after_id)
                self.after_id = None
            self.draw_tree()
            return

        self.animate = True
        self.current_depth = 1
        self.animate_btn.config(text="Сброс", bg="lightgreen")
        self.animate_step()

    def animate_step(self):
        if not self.animate:
            return

        self.draw_tree()

        if self.current_depth < self.depth:
            self.current_depth += 1
            self.after_id = self.root.after(
                self.animation_speed, self.animate_step
            )
        else:
            self.animate = False
            self.animate_btn.config(text="Анимация", bg="SystemButtonFace")
            self.after_id = None

    def update_animation_speed(self, value):
        self.animation_speed = int(value)

    def take_screenshot(self):
        try:
            self.root.update()
            # коэффициент масштабирования (DPI fix)
            scale = self.root.winfo_fpixels('1i') / 72
            x = int(self.root.winfo_rootx() * scale)
            y = int(self.root.winfo_rooty() * scale)
```

```
            w = int(self.root.winfo_width() * scale)
            h = int(self.root.winfo_height() * scale)

            img = ImageGrab.grab(bbox=(x, y, x + w, y + h))
            timestamp = time.strftime("%Y%m%d_%H%M%S")
            filename = f"pythagoras_tree_{timestamp}.png"
            img.save(filename)
            messagebox.showinfo("Скриншот", f"Сохранён как {filename}")
        except (OSError, IOError, AttributeError) as e:
            messagebox.showerror(
                "Ошибка", f"Не удалось создать скриншот: {str(e)}"
            )


if __name__ == "__main__":
    tk_root_window = tk.Tk()
    app = PythagorasTreeApp(tk_root_window)
    tk_root_window.mainloop()
```



Вывод:

Я освоил возможности языка программирования Python в разработке оконных приложений.